

MapReduce job mit Java in Eclipse

1. Einleitung

Eclipse kann (unter Nutzung der „Eclipse public license“) frei von www.eclipse.org für alle gängigen Plattformen heruntergeladen werden. Anbei ein paar Screenshots für Installation unter Linux. Es kann die aktuellste Version verwendet werden.

Um gradle als Buildsystem zu verwenden, ist es am einfachsten, das plugin für IntelliJ zu installieren. Für die Verwendung auf der Kommandozeile kann es nötig sein, den Installationspfad des gradle-plugins in den Pfad unter „Umgebungsvariablen“ aufzunehmen oder Gradle direkt von <https://gradle.org/releases/> herunterzuladen und zu installieren.

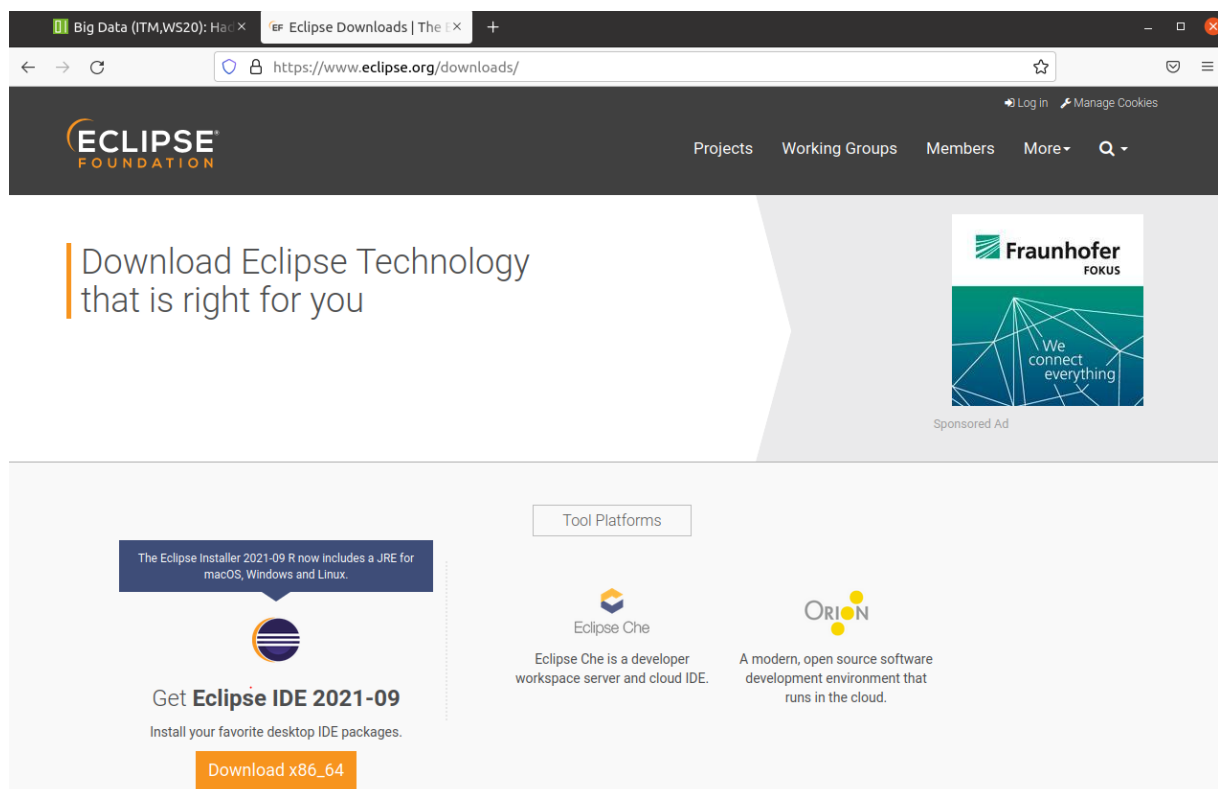
2. Generell

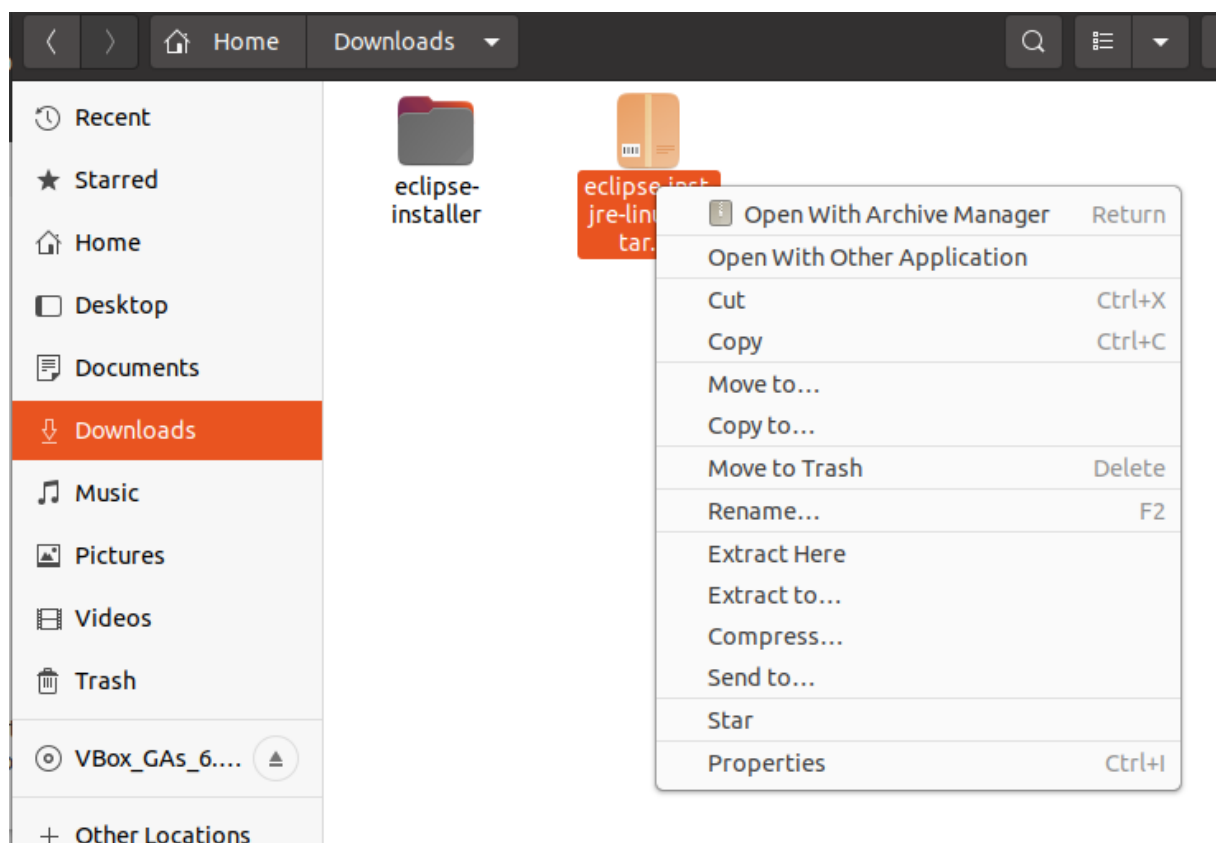
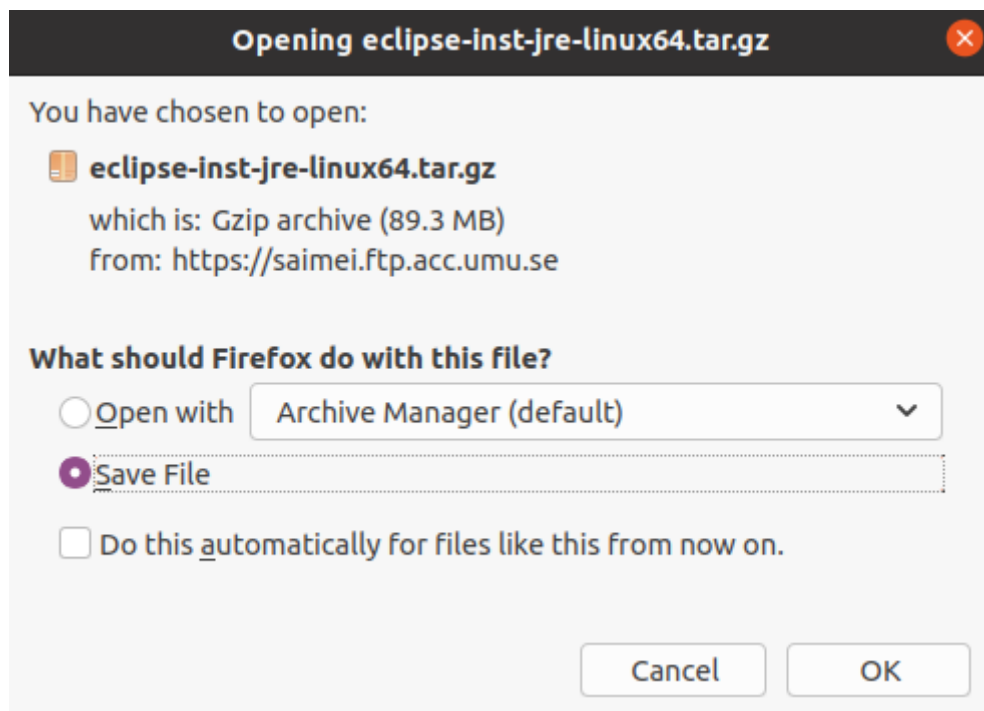
Folgende 2 Varianten bieten sich an, das Projekt zu bauen:

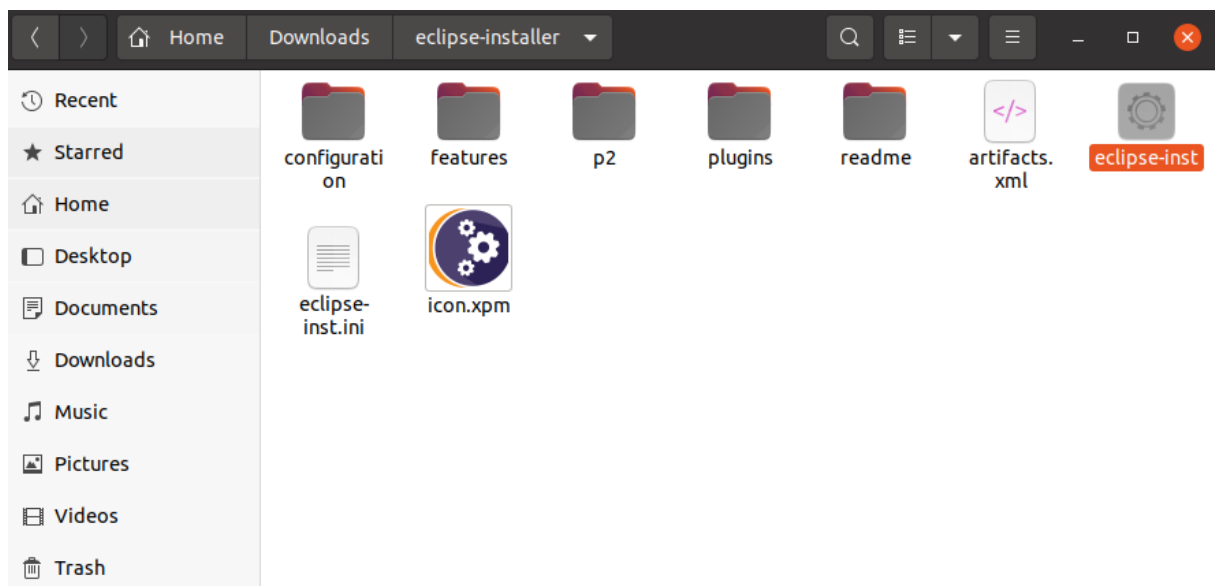
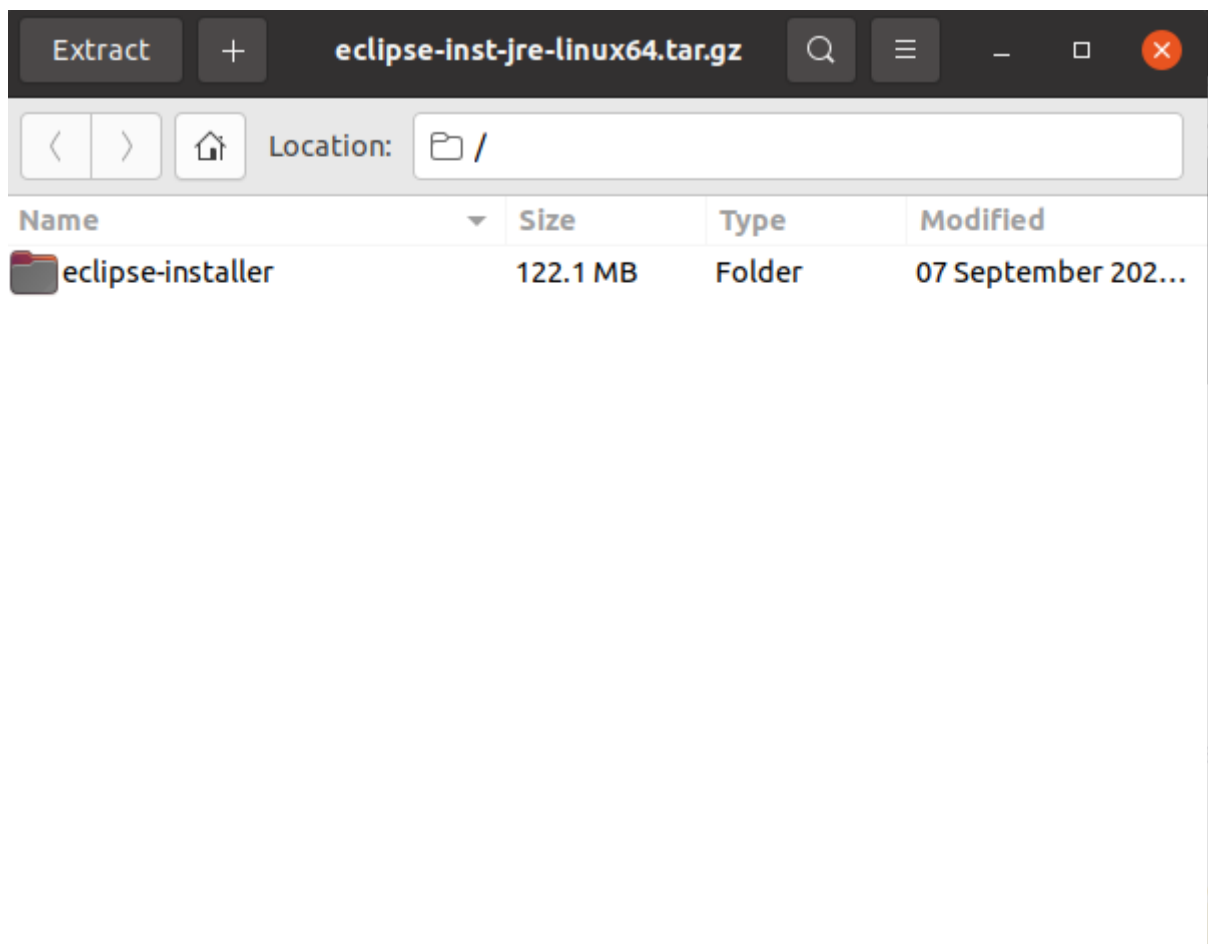
1. Mit Hilfe eines externen build systems (vorbereitetes build.gradle File)
2. Mit Hilfe von Eclipse selbst als Build-System (erfordert etwas mehr Vorbereitung, ist aber vielleicht besser zu verstehen)

Im Folgenden wird bei Abweichungen zwischen den Varianten stets die Notation @gradle bzw. @eclipse verwendet.

3. Installation







by Oomph

[★ DONATE](#)



type filter text



Eclipse IDE for Java Developers

The essential tools for any Java developer, including a Java IDE, a Git client, XML Editor, Maven and Gradle integration



Eclipse IDE for Enterprise Java and Web Developers

Tools for developers working with Java and Web applications, including a Java IDE, tools for JavaScript, TypeScript, JavaServer...



Eclipse IDE for C/C++ Developers

An IDE for C/C++ developers.



Eclipse IDE for Embedded C/C++ Developers

An IDE for Embedded C/C++ developers. It includes managed cross build plug-ins (Arm and RISC-V) and debug plug-ins (SEGGER...

Als Java Version muss eine Version ≥ 11 installiert werden.



Eclipse IDE for Enterprise Java and Web Developers

[details](#)

Tools for developers working with Java and Web applications, including a Java IDE, tools for JavaScript, TypeScript, JavaServer Pages and Faces, Yaml, Markdown, Web Services, JPA and Data Tools, Maven and Gradle, Git, and more.

Java 11+ VM

/usr/lib/jvm/java-1.11.0-openjdk-amd64



Installation Folder

/home/hduser/eclipse/jee-2021-092



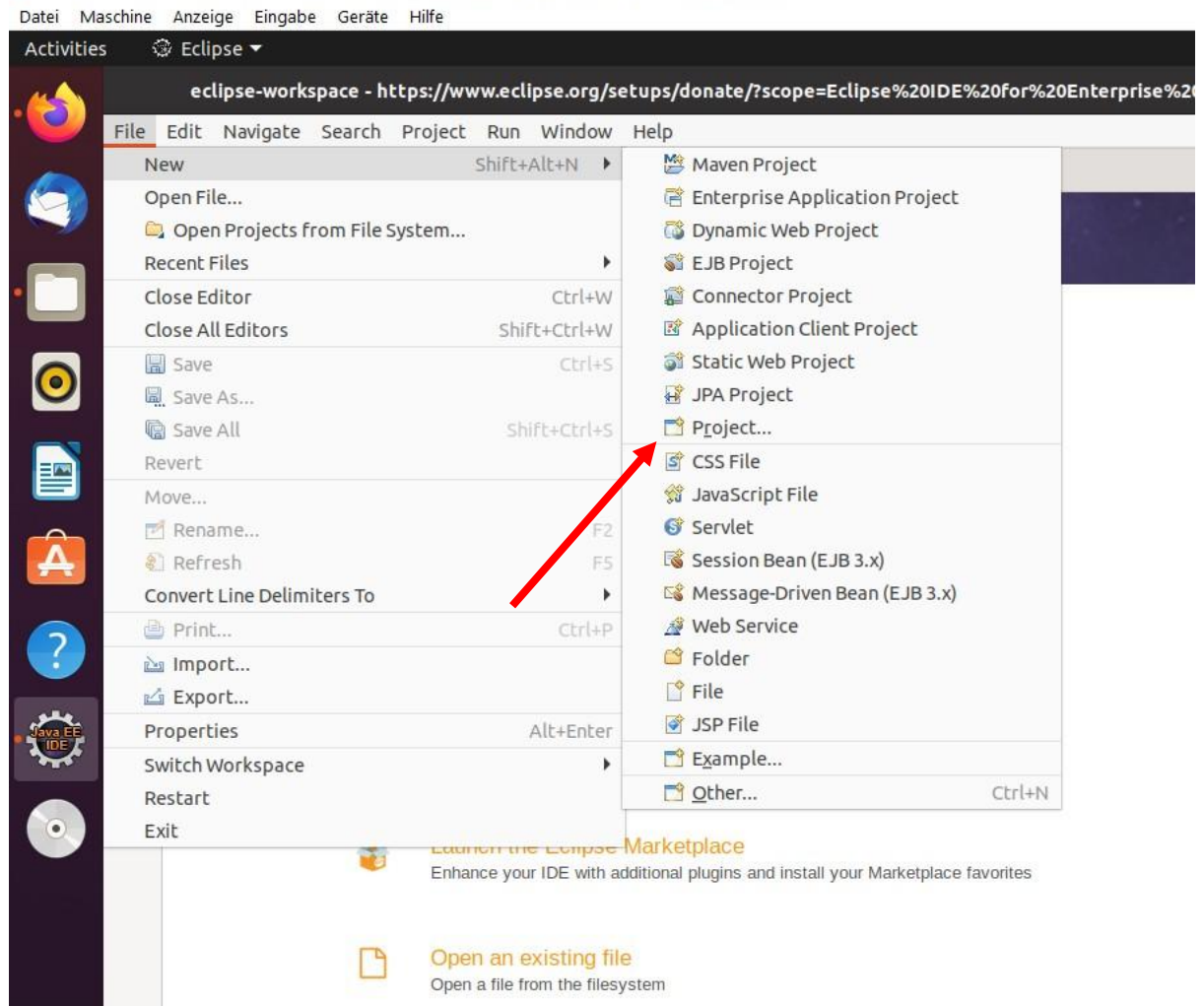
create start menu entry



create desktop shortcut

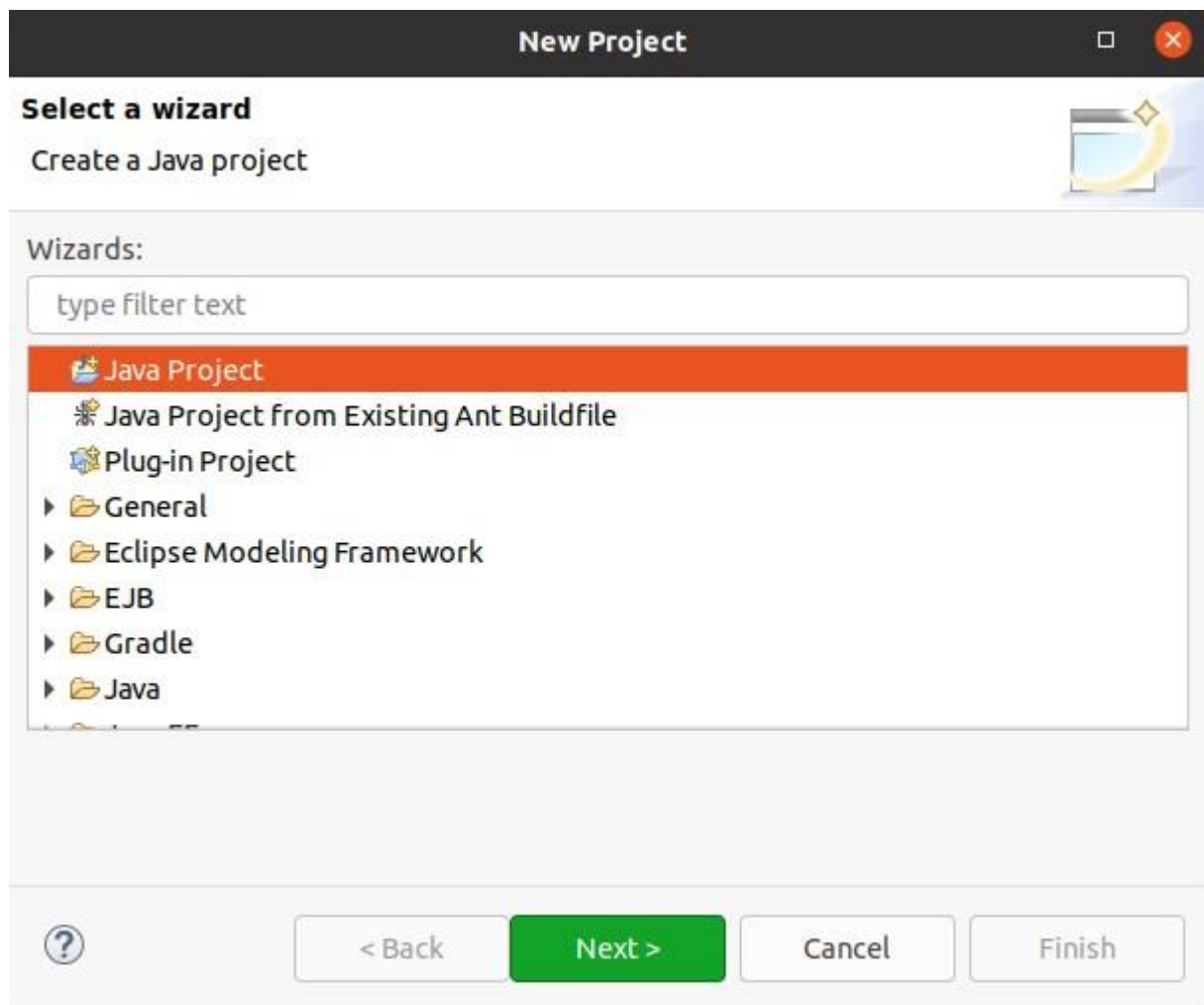
↓ INSTALL

← BACK



4. Project anlegen

@eclipse:



Wenn eine Java-Version > 11 (für hadoop < 3.5) bzw. Version > 17 (für hadoop 3.5+) verwendet wird, dann muss in Kompatibilität Version 11/17 gewählt werden.

New Java Project

Create a Java Project
Create a Java project in the workspace or in an external location.

Project name: itmWordCound

☒ Use default location
Location: /home/hduser/eclipse-workspace/itmWordCound [Browse...](#)

JRE

☒ Use an execution environment JRE: JavaSE-1.8
☐ Use a project specific JRE: java-11-openjdk-amd64
☐ Use default JRE 'java-11-openjdk-amd64' and workspace compiler preferences [Configure JREs...](#)

Project layout

☐ Use project folder as root for sources and class files
☒ Create separate folders for sources and class files [Configure default...](#)

Working sets

☐ Add project to working sets [New...](#)
Working sets: [Select...](#)

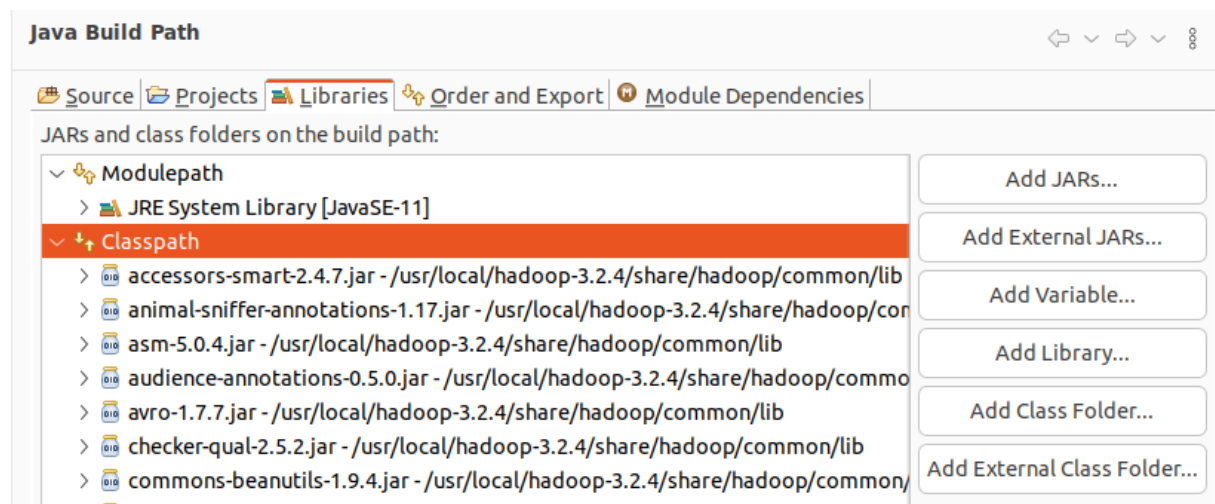
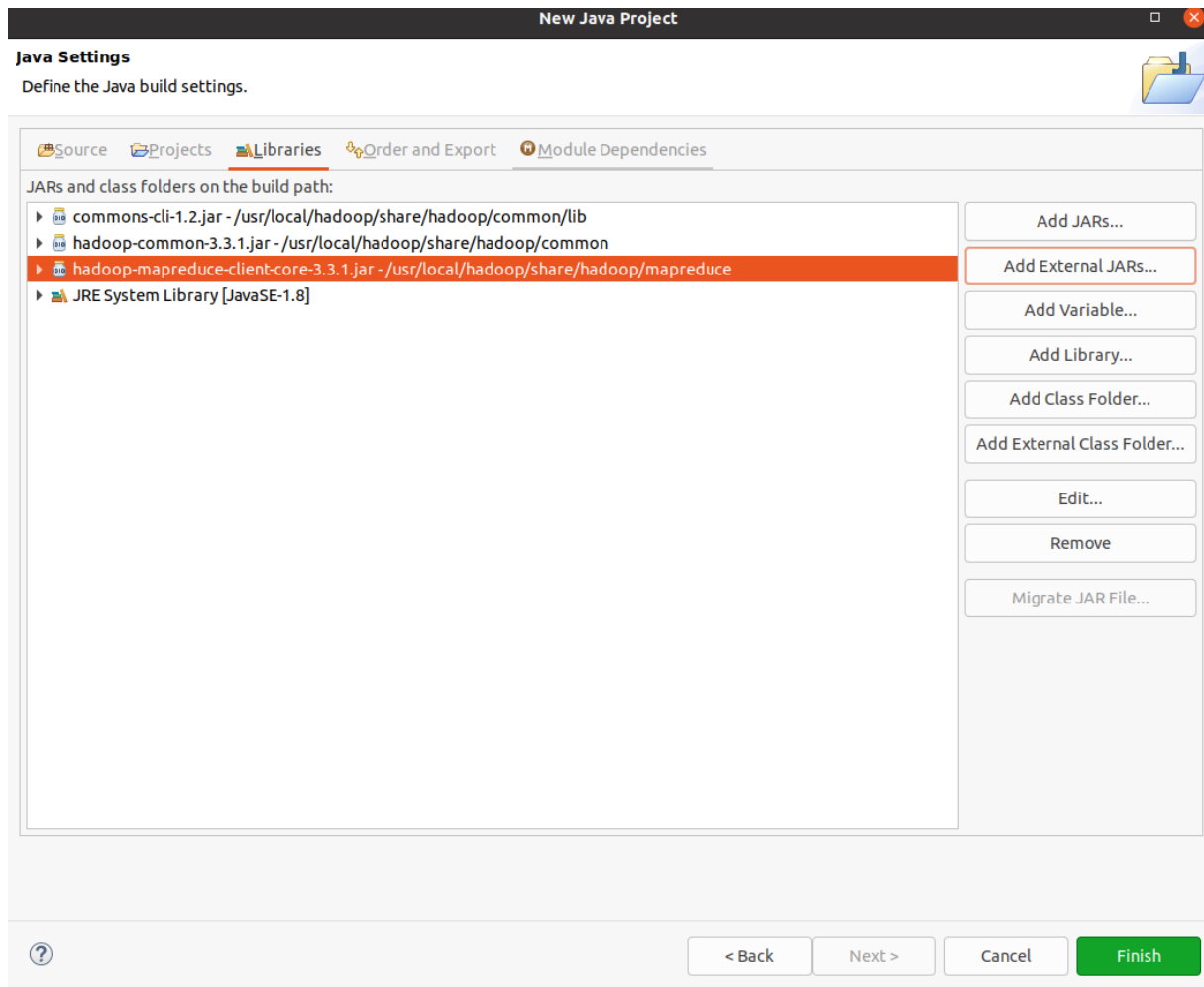
Module

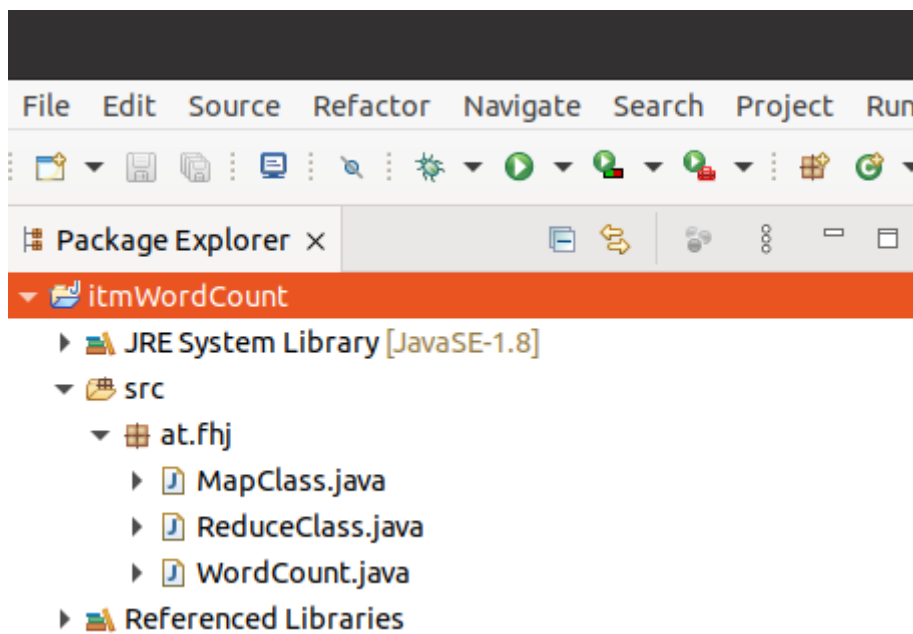
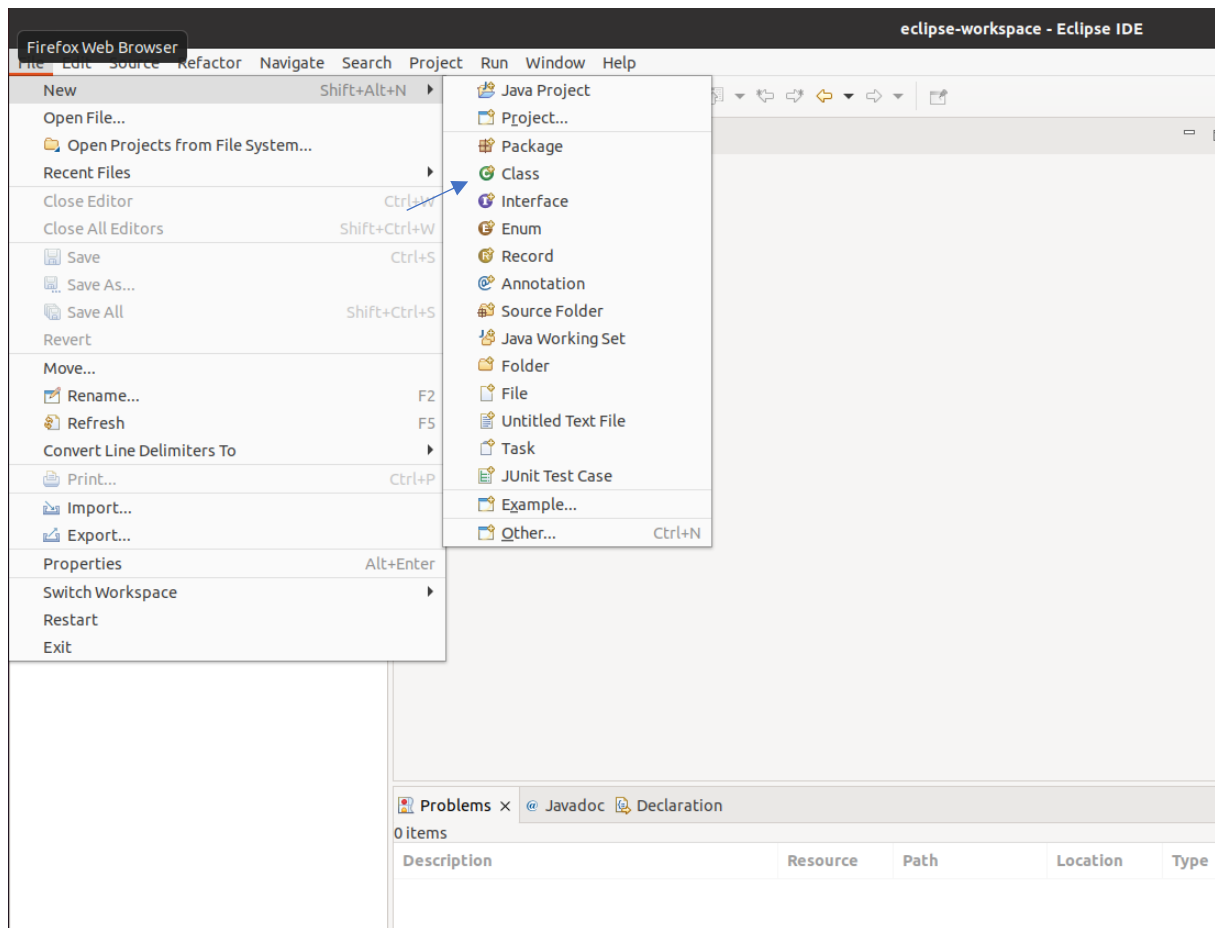
☐ Create module-info.java file

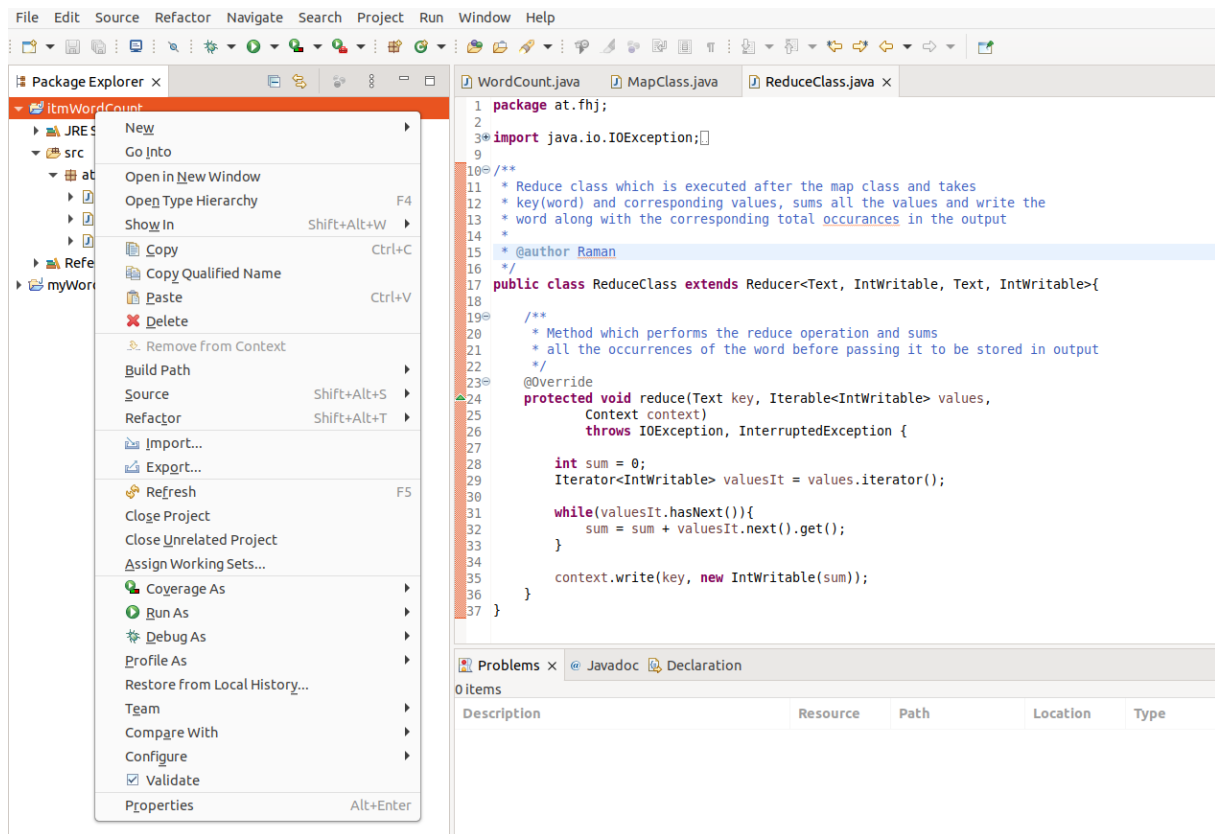
i The default compiler compliance level for the current workspace is 11. The new project will use a project specific compiler compliance level of 1.8. [Configure...](#)

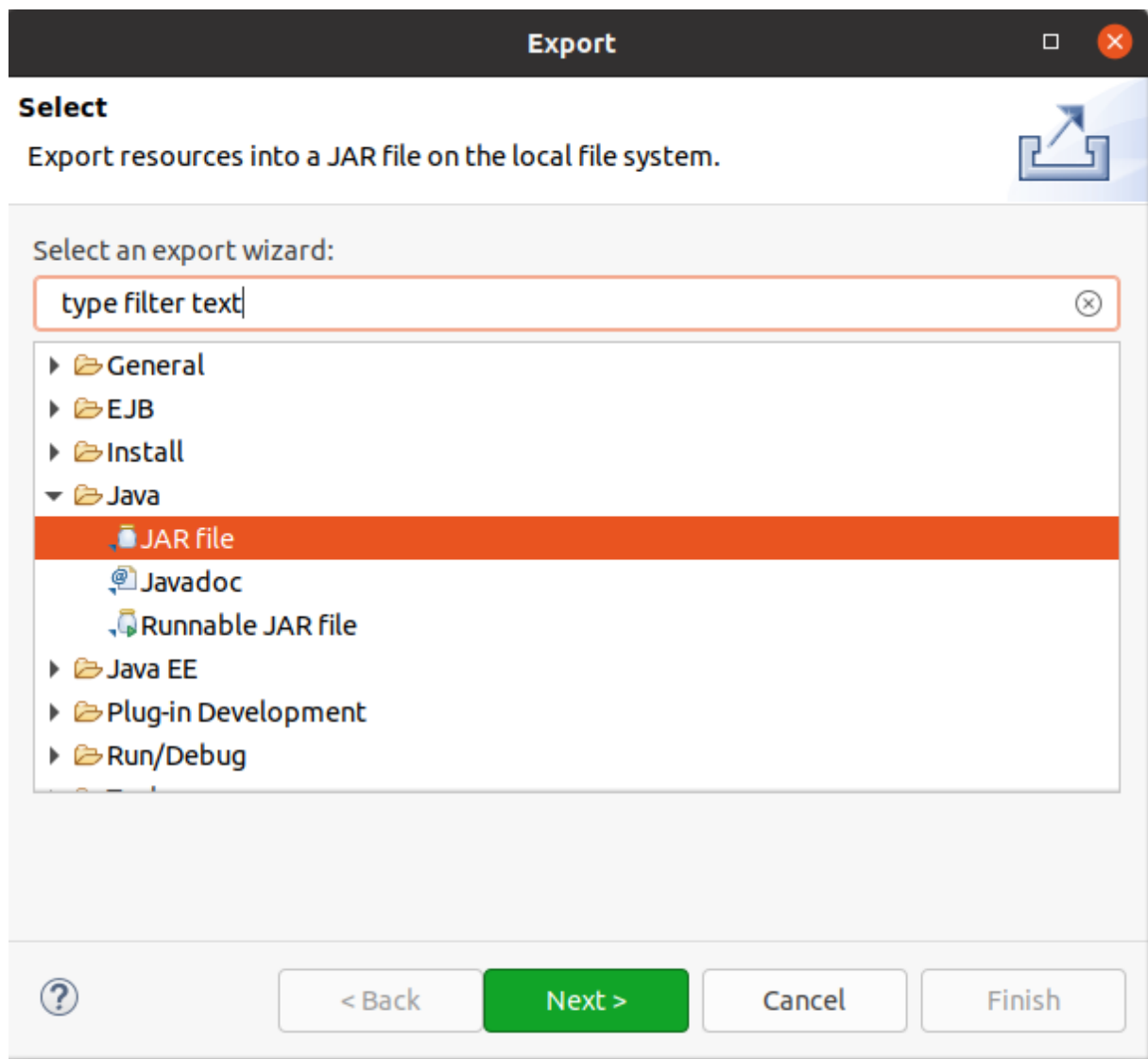
[?](#) [< Back](#) [Next >](#) [Cancel](#) [Finish](#)

Achtung: in neueren Eclipse-Versionen muss man die einzelnen Libraries unter "Classpath" dazuhängen, davor sind die Buttons ausgegraut.









Wichtig: Im folgenden Fenster nicht "Finish", sondern "Next" klicken, damit die weiteren Detailsangaben zu Manifest und v.a. Einsprungspunkt gesetzt werden können.

JAR Export

JAR File Specification

Define which resources should be exported into the JAR.

Select the resources to export:

☒ itmWordCount	☒ .classpath
☐ myWordCount	☒ .project

- ☒ Export generated class files and resources
- ☐ Export all output folders for checked projects
- ☐ Export Java source files and resources
- ☐ Export refactorings for checked projects. [Select refactorings...](#)

Select the export destination:

JAR file: Browse...

Options:

- ☒ Compress the contents of the JAR file
- ☐ Add directory entries
- ☐ Overwrite existing files without warning



< Back

Next >

Cancel

Finish

JAR Export



JAR Manifest Specification

Customize the manifest file for the JAR file.



Specify the manifest:

☒ Generate the manifest file

☐ Save the manifest in the workspace

☐ Use the saved manifest in the generated JAR description file

Manifest file:

Browse...

☐ Use existing manifest from workspace

Manifest file:

Browse...

Seal contents:

☐ Seal the JAR

Details...

☒ Seal some packages

Nothing sealed

Details...

Select the class of the application entry point:

Main class:

Browse...

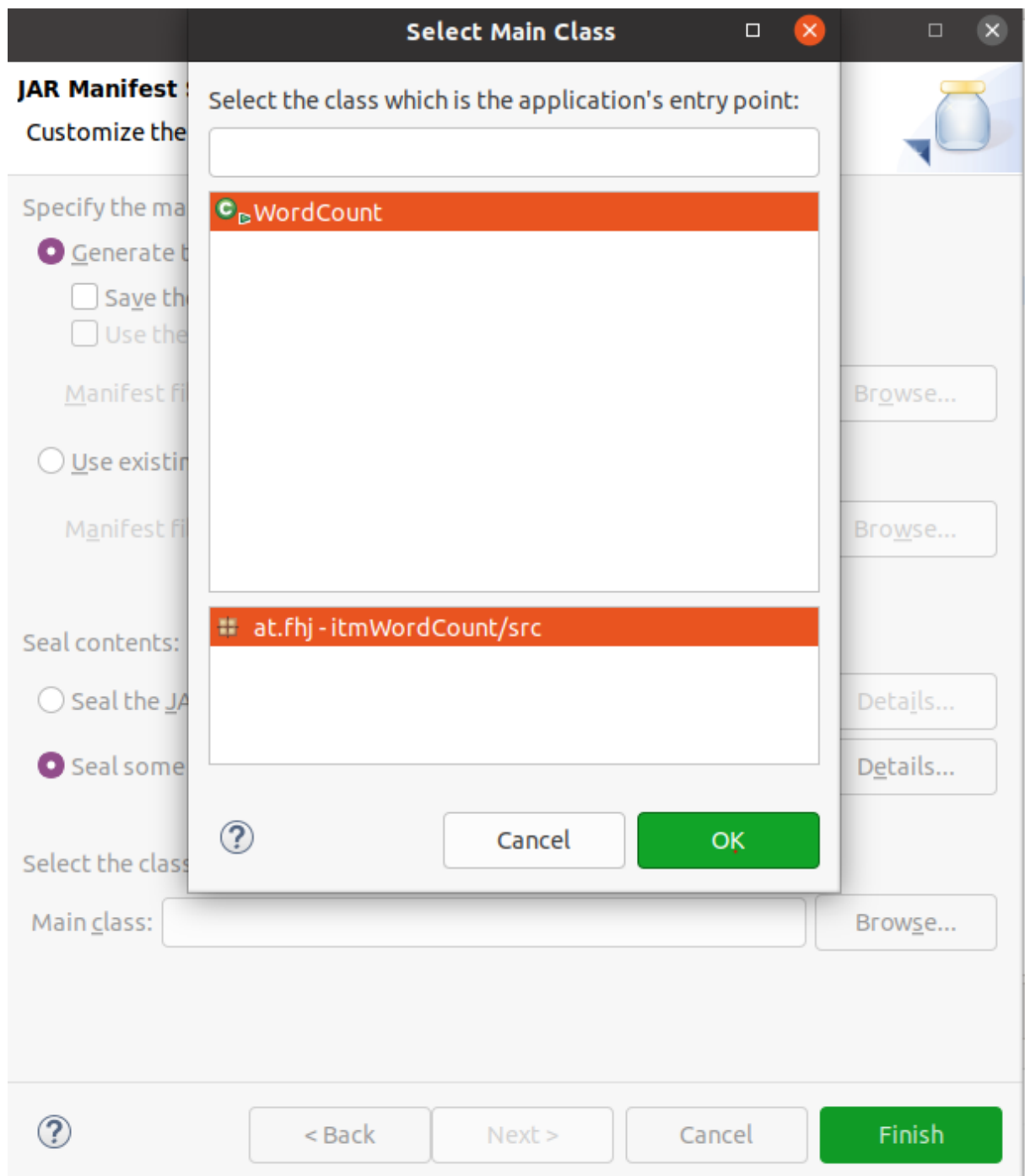


< Back

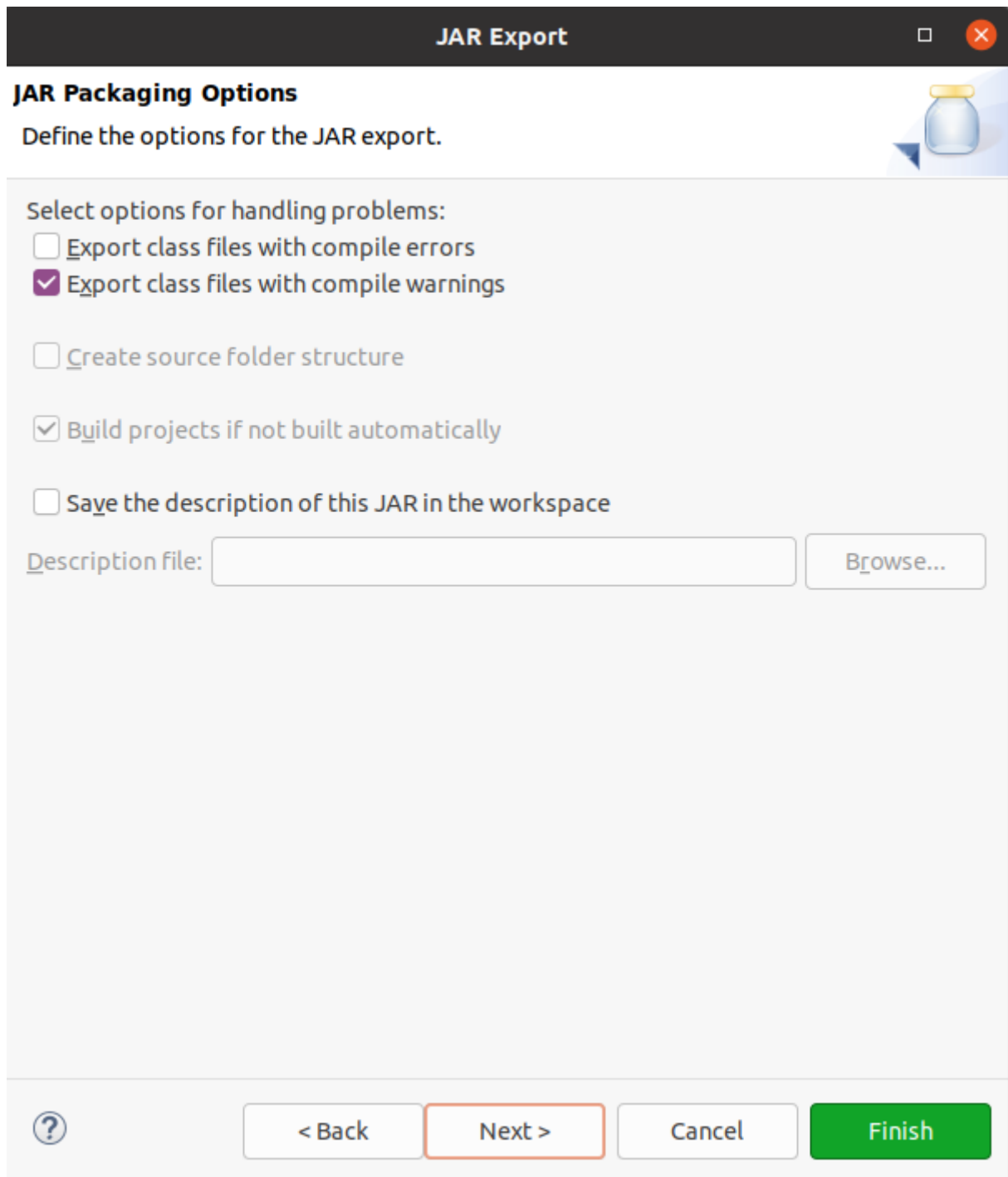
Next >

Cancel

Finish



Das Hakerl bei "Export class file with compile errors" sollte man entfernen, sonst gibt's erst Fehler zur Laufzeit.



@gradle:

1. Projekt als „Existing Gradle Project“ importieren

1. Menü **File** → **Import....**
2. Im Dialog:
 - Kategorie **Gradle** aufklappen.
 - Eintrag **Existing Gradle Project** wählen.
 - **Next**.

3. Bei „Project root directory“ den Ordner auswählen, in dem die **build.gradle** für das WordCount-Projekt liegt.
4. **Next** → Eclipse liest jetzt das Gradle-Projekt ein.
5. **Finish**.

Ergebnis:

- Im „Package Explorer“: Das Projekt sollte bereits die richtige Ordnerstruktur haben (**.java** Dateien unter **src/main/java/...**).
 - Wenn dies nicht der Fall, ist, dann einfach einen Source Folder anlegen und die Dateien dort hin verschieben: Rechtsklick auf Projekt → **New** → **Source Folder** → Name: **src/main/java**.
 - Damit die Einstellungen übernommen werden: Rechtsklick auf das Projekt → **Gradle** → **Refresh Gradle Project**.
- Die Hadoop-Bibliotheken werden automatisch über die **dependencies** in der **build.gradle** übernommen – man muss sie nicht manuell als External JARs hinzufügen.

2. Projekt über den Gradle Tasks View bauen und Jar erstellen

1. Menü **Window** → **Show View** → **Other...**
2. Kategorie **Gradle** → **Gradle Tasks**.
3. In der „Gradle Tasks“-Ansicht das Projekt auswählen.
4. Unter der Gruppe **build** entweder **build** oder **jar** wählen.

Standardmäßig erzeugt Gradle das Jar unter dem Pfad **build/libs/<projektname>-<version>.jar**. In diesem Fall unter **build/libs/Hadoopwordcount-1.0.jar**

6. Prüfung des erstellten Artifacts und Kopieren auf Zielplattform

Je nach verwendetem Buildsystem und deren Settings findet man das generierte Jar-File unter z.B. **out/artifacts** oder **build/libs** und beinhaltet der Dateiname eine Versionsnr. oder nicht.

Das **build.gradle** beinhaltet einen „**deploy**“ Task, dieser funktioniert jedoch nur bei Passwort-based login bzw. müsste man die Credentials und Hostnamen ändern.

Daher am besten händisch die Datei manuell kopieren als user „student“ und dann die Datei dem hduser „schenken“:

```
scp -I <PrivateKeyFile> Hadoopwordcount-1.0.jar
student@<hostname>:/tmp/Hadoopwordcount-1.0.jar
ssh student@<hostname>
sudo chown hduser:hadoop /tmp/Hadoopwordcount-1.0.jar
su - hduser
mv /tmp/Hadoopwordcount-1.0.jar ~/Hadoopwordcount.jar
```

7. Test des generierten jar-Files

Hadoop starten und Test aufrufen mit dem zuvor generierten und auf die Zielplattform kopierten Jar-File.

Job ausführen

- 1.) Input Ordner und Beispiel Textdatei im hdfs anlegen
- 2.) Job ausführen (Vorsicht Pfade sind ggf. anzupassen!)

```

su - hduser
start-dfs.sh
hdfs dfs -mkdir /input
hdfs dfs -mkdir /output
hdfs dfs -put ~/BigData/data/Bibel.txt /input/
OutputDir=/output/Bibel
# folgendes für jeden Versuch neu ausführen
hdfs dfs -rm -R $OutputDir
hadoop jar Hadoopwordcount.jar /input/Bibel.txt $OutputDir

```

Bei Problemen (vor allem wenn Meldung „main class not found“) die Jar-Datei überprüfen ("jar -tvf" funktioniert unter Linux als auch Windows), sie muss zumindest folgende 4 Dateien beinhalten:

```

jar -tvf Hadoopwordcount.jar | awk '{ print $8 }'
META-INF/MANIFEST.MF
at/fhj/MapClass.class
at/fhj/ReduceClass.class
at/fhj/WordCount.class

```

Der Inhalt der Datei MANIFEST.MF sollte wie folgt sein (optional stehen dahinter noch ClassPaths):

```

jar -xf Hadoopwordcount.jar META-INF/MANIFEST.MF
cat META-INF/MANIFEST.MF
Manifest-Version: 1.0
Main-Class: at.fhj.WordCount

```

Erwarteter Output nach erfolgreichem MapReduce Job wie folgt:

```

hduser@osboxes: ~/Desktop
Reduce input groups=56030
Reduce shuffle bytes=9199249
Reduce input records=781853
Reduce output records=56030
Spilled Records=1563706
Shuffled Maps =1
Failed Shuffles=0
Merged Map outputs=1
GC time elapsed (ms)=55
Total committed heap usage (bytes)=706740224
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=4579278
File Output Format Counters
  Bytes Written=614705
Job was successful
hduser@osboxes:~/Desktop$

```

Im angegebenen Output-Verzeichnis befinden sich dann 2 Dateien, eine leere Datei "_SUCCESS" und eine Datei mit dem Ergebnis des Jobs (wäre der Output größer als konfigurierte Blockgröße würde es weitere Dateien part-r-00001 usw. geben).

localhost:9870/explorer.html#/output/bibel

Hadoop Overview Datanodes Datanode Volume Failures Snapshot Startup Progress Utilities

Browse Directory

/output/bibel Go!

Show 25 entries Search:

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	hduser	supergroup	0 B	Nov 04 15:18	2	16 MB	_SUCCESS
-rw-r--r--	hduser	supergroup	363.59 KB	Nov 04 15:18	2	16 MB	part-r-00000

File information - part-r-00000

Download Head the file (first 32K) Tail the file (last 32K)

Block information -- Block 0

Block ID: 1073741956
Block Pool ID: BP-109114966-127.0.1.1-1698821495461
Generation Stamp: 1138
Size: 372320
Availability:
• UbuntuBigData

File contents

```
mme      1
verstummen  1
verstummt   2
verstummt   1
```

Das "Tail the file" funktioniert üblicherweise nicht über die GUI, daher besser über Kommandozeile:

```
hdfs dfs -tail /output/Bibel/part-r-00000
hdfs dfs -head /output/Bibel/part-r-00000
```

Wichtig: wenn der Job erneut gestartet werden soll, muss zuvor das Output-Verzeichnis gelöscht werden (sowohl wenn man Output ins lokale Dateisystem als auch wenn man dies in hdfs schreibt)!

8. Optional: Nur wenn yarn konfiguriert und verwendet wurde

Auf der Webseite von Yarn sieht man die erledigten Jobs ebenfalls (zu sehen nur im Fall, wenn mapreduce.framework.name auf „yarn“ und nicht „local“ eingestellt wurde):



Cluster

- About
- Nodes
- Node Labels
- Applications
 - NEW
 - NEW SAVING
 - SUBMITTED
 - ACCEPTED
 - RUNNING
 - FINISHED
 - FAILED
 - KILLED
- Scheduler

Tools

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running
1	0	0	1	0

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned
3	0	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation
Capacity Scheduler	[memory-mb (unit=Mi), vcores]	<memory:1024, vCores:1>	<memory:8192, vCores:4>

Show 20 entries

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime
application_1763115893210_0003	hduser	my word count	MAPREDUCE		root.default	0	Fri Nov 14 11:31:47 +0100 2025	Fri Nov 14 11:31:49 +0100 2025

Showing 1 to 1 of 1 entries